

AI Sponsor

By Kyle Mard

Background

- SipSmarter app
- Firebase Firestore
- Cloud Functions
- AI Agents



Problem Statement

- Existing solutions typically present raw data without meaningful interpretation, making it difficult to recognize patterns, assess risk, or make informed behavioral changes.

Basic Idea

- Aggregates raw data into meaningful metrics
- Uses a RAG approach to incorporate health-related knowledge
- Applies an AI model to interpret user behavior and generate insights

Demonstration



SignUp

Login

Email

Password



Confirm Password



Create Account

or

Continue as Guest

Code

- Knowledge Base ->

```
knowledge = [  
    "Drinking more than 14 drinks per week increases long-term health risks.",  
    "Alcohol calories contribute significantly to weight gain.",  
    "Spacing drinks over time reduces intoxication and health impact.",  
    "Frequent drinking patterns can indicate habit formation.",  
    "Weekend binge drinking is associated with higher health risks.",  
    "Gradually reducing alcohol intake is more sustainable than stopping abruptly."  
]
```

- Embeddings ->

```
embedded_knowledge = []  
  
def init_embeddings():  
    global embedded_knowledge  
  
    if embedded_knowledge:  
        return  
  
    for text in knowledge:  
        emb = client.embeddings.create(  
            model="text-embedding-3-small",  
            input=text  
        )  
        embedded_knowledge.append({  
            "text": text,  
            "embedding": emb.data[0].embedding  
        })
```

```
query_emb = client.embeddings.create(  
    model="text-embedding-3-small",  
    input=summary_text  
)  
query_emb = query_emb.data[0].embedding
```

- Retrieved Context (what RAG selects) ->

```
scored = [  
    (cosine_similarity(query_emb, item["embedding"]), item["text"])  
    for item in embedded_knowledge  
]  
  
scored.sort(reverse=True)  
  
top_context = [text for _, text in scored[:3]]
```

- Inject retrieved context (where RAG is applied) ->

```
response = client.responses.create(  
    model="gpt-4o-mini",  
    input=f"""  
    User drinking summary:  
    {summary_text}  
  
    Relevant guidance:  
    {context}  
  
    Provide:  
    - One insight  
    - One actionable recommendation  
    """,  
    )
```

Evaluation

- What was most challenging?
 - Implementing RAG in conjunction with Firestore, cloud functions, and OpenAI API
- What can be improved?
 - Personalized RAG using user history
 - Scalable vector database